MODULE 16 · DOMAIN 2 - SECURITY

Roles, Temporary Credentials and the Credential Chain

How code actually obtains the permissions module 15 described - and why it should never hold a password.

DOMAIN 2 of 4 EXAM WEIGHT 26% MODULE 16 of 34

admin@sthithapragna.com

An application should never **hold** credentials. It should be **given** credentials, automatically, that expire.

What temporary credentials are

- An **access key ID**, a **secret access key**, and crucially a **session token**.
- They carry an **expiration**. After it, every call fails until they are refreshed.
- The session token is what distinguishes them. Long-lived user keys have only two parts.
- They are issued by **AWS STS**, and they are not stored anywhere in IAM.
- Nothing needs revoking when they leak, because they expire on their own.

> ON THE EXAM
>
> The security token included in the request is invalid usually means the session token was dropped. Copying only the two keys from a set of three is a classic mistake.

Long-lived keys or a role?

IAM USER ACCESS KEY

Two parts, no expiry

- Valid until you delete it
- Must be stored somewhere
- Rotation is a manual process
- Leaks are permanent until noticed
- Appropriate for almost nothing now

ASSUMED ROLE

Three parts, expiring

- Expires in an hour by default
- Delivered at runtime, never stored
- Rotates with no action from you
- A leak expires by itself
- The answer for every workload

THE TIEBREAKER

On this exam, any option that creates or stores an IAM user access key is essentially always wrong. The only real exception is a system that cannot federate at all.

The STS operations

OPERATION	USE IT WHEN
AssumeRole	An AWS identity assumes a role. Same account or cross-account
AssumeRoleWithWebIdentity	A user has signed in with an OIDC provider such as Google or Cognito
AssumeRoleWithSAML	A user has authenticated with a SAML 2.0 identity provider
GetSessionToken	An IAM user needs temporary credentials, typically with MFA
GetFederationToken	A proxy application issues credentials to non-AWS users
GetCallerIdentity	Ask AWS who you currently are. Needs no permissions at all

ON THE EXAM

GetCallerIdentity is the fastest debugging tool in AWS. It answers 'which identity is this code actually running as', which is often the real question.

AssumeRole in detail

- The caller needs **sts:AssumeRole**, and the role's **trust policy** must allow the caller.
- STS returns temporary credentials with the permissions of the **role**, not the caller.
- The caller's own permissions are **replaced**, not added to. Assuming a role can reduce your access.
- **RoleSessionName** appears in CloudTrail, so it is how you attribute activity to a person.
- A **session policy** passed at assume time further restricts the session. It cannot expand it.

> ON THE EXAM
>
> The replacement behaviour surprises people. Assuming a role is switching identity, not gaining an extra one, and you keep nothing from before.

Session durations

OPERATION	DURATION
AssumeRole	15 minutes to 12 hours. Default 1 hour
Role maximum session duration	Set on the role itself, up to 12 hours. It caps the request
Role chaining	1 hour maximum. Requesting more fails outright
GetSessionToken, IAM user	15 minutes to 36 hours. Default 12 hours
GetSessionToken, root user	15 minutes to 1 hour
AssumeRoleWithWebIdentity	15 minutes to 12 hours. Default 1 hour

ON THE EXAM

Two caps apply to AssumeRole: what you request, and the role's own maximum session duration. Requesting more than the role permits fails.

Role chaining

- Using credentials from one assumed role to assume a second role is **role chaining**.
- **The second session is capped at 1 hour**, whatever the roles allow.
- Requesting a longer duration returns an error rather than a shorter session.
- The usual fix is to grant the first role the permissions directly, avoiding the second hop.
- It happens accidentally: a Lambda or Fargate task assuming a cross-account role is already chaining.

> ON THE EXAM
>
> This is the classic 'works on my laptop, fails in production' bug. Locally you assume from an IAM user, which is not chaining. In Lambda you assume from a role, which is.

Assuming a role in code

- Create an **STS client**, call AssumeRole, and build a second client from the returned credentials.
- Most SDKs offer a **credentials provider** that does this and refreshes automatically.
- Calling AssumeRole on every request is wasteful. Cache the credentials until near expiry.
- In the CLI and SDKs, a profile with **role_arn** and **source_profile** does it declaratively.
- Never write the returned credentials to disk or to an environment variable that persists.

> ON THE EXAM
>
> Reach for the SDK's assume-role credentials provider rather than calling AssumeRole yourself. It handles caching and refresh, which hand-written code usually gets wrong.

The default credential provider chain

01

Environment

Java system properties, then AWS_ACCESS_KEY_ID and friends.

02

Web identity

A token file, used by EKS and other OIDC runtimes.

03

Shared files

The profile in ~/.aws/credentials and ~/.aws/config.

04

Container

The ECS or EKS container credentials endpoint.

05

Instance

The EC2 instance profile, read from instance metadata.

ON THE EXAM

Order matters, because the first provider that returns credentials wins. A stray environment variable silently overrides the instance role.

Why the chain matters

- Correctly written code needs **no credential configuration at all**. It works everywhere.
- Explicitly passing keys to a client constructor **bypasses the chain entirely**. Almost always wrong.
- A leftover environment variable or profile beats the instance role, silently.
- **GetCallerIdentity** tells you which provider actually won.
- The SDK **refreshes** temporary credentials before they expire, without your code doing anything.

> ON THE EXAM
>
> That last point is why long-running processes do not need refresh logic. Writing your own refresh loop is a sign the chain is being bypassed.

Where each service gets its credentials

RUNTIME	DELIVERY MECHANISM
AWS Lambda	Environment variables, set by the service from the execution role
Amazon EC2	Instance metadata at 169.254.169.254, from the instance profile
Amazon ECS	A container credentials endpoint, from the task role
Amazon EKS	A projected web identity token, exchanged via AssumeRoleWithWebIdentity
AWS CodeBuild	The build project's service role
On-premises servers	IAM Roles Anywhere, using an X.509 certificate

ON THE EXAM

Every one of these delivers temporary, auto-rotating credentials. None of them requires a stored access key, which is why storing one is always the wrong answer.

Instance metadata and IMDSv2

- Metadata is served at **169.254.169.254**, a link-local address reachable only from the instance.
- Role credentials sit under **iam/security-credentials/** and rotate automatically.
- **IMDSv1** answered any GET request, so an SSRF bug in your app could read credentials.
- **IMDSv2** requires a PUT to obtain a session token first, which most SSRF attacks cannot do.
- IMDSv2 can be **required** per instance, and Amazon Linux 2023 enforces it by default.

ON THE EXAM

Requiring IMDSv2 is a standard hardening answer. The attack it prevents is a server-side request forgery in your own application reading the instance role's credentials.

The trust policy, revisited

- A resource-based policy on the role. Its **Principal** names who may assume it.
- For a service, the principal is a **service principal** such as lambda.amazonaws.com.
- For cross-account, it is an account ARN or a specific role ARN in the other account.
- Naming an account root trusts **anything in that account** that also has sts:AssumeRole.
- Naming a specific role ARN is tighter, and is the least-privilege choice.

> ON THE EXAM
>
> Trusting an account root is not the same as trusting one principal in it. The other account's administrators decide who gets to use that trust.

The confused deputy problem

- A third party assumes a role in your account to do work for you. So far so good.
- If they know another customer's role ARN, they could be tricked into acting against that account.
- **ExternalId** is a secret value the trust policy requires, unique to each customer.
- You supply it to the third party, and they must present it when assuming the role.
- For AWS services calling on your behalf, use **aws:SourceArn** and **aws:SourceAccount** instead.

> ON THE EXAM
>
> ExternalId is for third-party SaaS vendors. SourceArn and SourceAccount are the equivalent when an AWS service is the deputy.

SAML or web identity federation?

SAML 2.0

Corporate identity

- Active Directory, Okta, Ping
- AssumeRoleWithSAML
- Employees signing in to AWS
- Trust is a SAML identity provider
- Console access via the SAML endpoint

WEB IDENTITY, OIDC

Consumer or workload identity

- Google, Facebook, Apple, Cognito
- AssumeRoleWithWebIdentity
- Mobile and web app end users
- Trust is an OIDC provider
- Also how EKS and GitHub Actions work

THE TIEBREAKER

Employees means SAML. App users means web identity. Both end in temporary credentials from a role, so neither ever needs an IAM user.

Web identity in practice

- A mobile app cannot hold AWS credentials. Anything shipped to a device is readable.
- The user signs in to an identity provider and receives a token.
- The app exchanges that token for temporary AWS credentials via a role.
- The role's policy can use **policy variables** so each user reaches only their own data.
- **Amazon Cognito identity pools** package this whole exchange. Module 17 covers it.

> ON THE EXAM
>
> Any option that puts AWS access keys into a mobile app, a browser or a public repository is wrong. Web identity federation is the standard alternative.

admin@sthithapragna.com

IAM Identity Center

- Central sign-in for people across every account in an AWS Organization.
- **Permission sets** become roles provisioned automatically into each account.
- Identity comes from its own directory, or from Active Directory, or from an external provider.
- It replaces creating IAM users in each account, which does not scale and is hard to offboard.
- It is for **humans**. Applications still use roles attached to compute.

> ON THE EXAM
>
> The exam's framing: IAM users are a legacy pattern for people. Identity Center or federation is the modern answer, and a role is always the answer for code.

Service-linked roles

- A role predefined by a service, linked to it, that the service assumes to act on your behalf.
- Created automatically when you first use the feature that needs it.
- You **cannot edit its permissions**, and you cannot delete it while the service still needs it.
- Examples: Auto Scaling, Elastic Load Balancing, RDS and many others.
- Distinct from a **service role**, which you create and control yourself.

> ON THE EXAM
>
> The distinction: a service role is yours to define, a service-linked role is AWS's. If a scenario asks to modify a role's permissions and cannot, it is service-linked.

IAM Roles Anywhere

- Gives temporary AWS credentials to workloads running **outside** AWS.
- Authentication is an **X.509 certificate** from a certificate authority you register as a trust anchor.
- The server exchanges its certificate for temporary credentials from STS.
- This removes long-lived IAM user access keys from on-premises servers.
- It requires a PKI. If you have none, that is a real prerequisite rather than a detail.

> ON THE EXAM
>
> When a scenario says on-premises servers need AWS access without stored access keys, this is the intended answer.

Revoking a session

- Temporary credentials **cannot be deleted**. There is nothing stored to remove.
- Instead, attach an inline policy denying everything when **aws:TokenIssueTime** is before now.
- The console offers this as **Revoke active sessions** on the role.
- Existing sessions stop working immediately. New assumptions still succeed.
- For a leaked long-lived access key, the answer is different: deactivate and delete the key.

> ON THE EXAM
>
> This is the practical answer to 'temporary credentials have leaked and expire in eleven hours'. Waiting is not the only option.

Session tags and source identity

- **Session tags** are key-value pairs passed when assuming a role.
- They surface as **aws:PrincipalTag**, which is what makes ABAC work with federated users.
- **SourceIdentity** records who the original human was, and cannot be changed once set.
- It persists through role chaining, so attribution survives multiple hops.
- Both appear in CloudTrail, which is how shared-role activity is traced to a person.

> ON THE EXAM
>
> Module 15's ABAC needs principal tags. For a federated user there is no IAM identity to tag, so session tags are where those tags come from.

How requests are actually signed

- Every AWS API call is signed with **SigV4**, using the secret key and the session token.
- The signature covers the method, URI, headers, payload hash, timestamp and Region.
- **The secret key is never transmitted.** Only the derived signature is.
- Clock skew beyond about 5 minutes causes signature failures. A real cause of mystery 403s.
- The SDKs do all of this. You only sign manually when calling AWS from something without an SDK.

> ON THE EXAM
>
> The clock skew detail is worth remembering: a container or device with a wrong clock fails every AWS call, and the error mentions the signature rather than the time.

Auditing who did what

- CloudTrail records the **assumed-role ARN** and the **role session name** on every call.
- A shared role with a generic session name makes activity **unattributable**.
- Set a meaningful **RoleSessionName**, such as the user's identifier, and it appears in every event.
- **SourceIdentity** is stronger, because it cannot be changed once set.
- The AssumeRole call itself is also logged, so you can trace a session back to its origin.

> ON THE EXAM
>
> Roles solve the credential problem and create an attribution problem. Session names and source identity are how you solve the second one.

Ask the same question each time: **what identity does this thing already have**, and what is it trying to become?

> A mobile photo app lets users upload images to Amazon S3. Users sign in with Google. Each user must be able to read and write only objects under their own prefix in a shared bucket.
>
> The current build embeds an IAM user's access keys in the application. Security has flagged this and it must be removed.

How should the app obtain AWS access?

A Store the access keys in the device keychain, encrypted, rather than in the binary

B Exchange the Google token for temporary credentials via a role, using web identity federation, with a policy scoped by policy variables

C Create an IAM user per app user and issue each one its own access keys

D Route all uploads through an EC2 proxy that holds the access keys

◆ PAUSE AND DECIDE

How should the app obtain AWS access?

A Keys in the keychain

Still ships credentials to a device you do not control. Encryption at rest does not help when the app must decrypt them to use them.

B Web identity federation

No credentials are shipped at all. The user's existing Google sign-in is exchanged for short-lived credentials, and policy variables scope each user to their own prefix.

C An IAM user per app user

IAM users are for a handful of long-lived identities, not for application end users. It also hits the 5,000 user quota quickly.

D EC2 proxy

It does remove keys from the device, at the cost of an always-on server carrying every byte of every upload. Module 9 rejected this shape already.

THE SIGNAL

Every option except B involves credentials that persist. Federation is the only pattern where nothing long-lived exists to steal.

> An ECS task on Fargate assumes a cross-account role to read from a partner's S3 bucket. The role's maximum session duration is set to 12 hours, and the code requests 12 hours. Locally, a developer running the same code with their own IAM user credentials gets a 12-hour session successfully.
>
> In Fargate the AssumeRole call fails outright with an error about the requested duration.

What explains the difference?

A Fargate tasks cannot assume cross-account roles

B The role's trust policy is missing the ECS service principal

C The task already runs as a role, so this is role chaining, which caps sessions at 1 hour and fails if more is requested

D The task execution role lacks sts:AssumeRole permission

◆ PAUSE AND DECIDE

What explains the difference?

A Fargate cannot assume cross-account roles

It can, and does so routinely. Nothing about Fargate prevents cross-account assumption.

B Missing service principal

That would prevent assumption entirely with an access denied error, not an error about duration. The error names the duration.

C Role chaining caps at 1 hour

The task runs as its task role, so assuming another role is chaining. That caps at 1 hour, and requesting more fails rather than truncating.

D Missing sts:AssumeRole

Also produces access denied rather than a duration error. And the call clearly reaches STS, since STS is what rejects the duration.

THE SIGNAL

Two clues: the error is specifically about duration, and it works from an IAM user but not from a role. Only chaining explains both.

> A SaaS monitoring vendor needs read access to its customers' AWS accounts. Each customer creates a role that the vendor's account assumes. The vendor holds thousands of such role ARNs.
>
> A security review notes that if an attacker learns another customer's role ARN and can persuade the vendor's platform to use it, the vendor's legitimate credentials would satisfy that trust policy too.

What should the trust policy require?

A. A condition on sts:ExternalId, with a unique secret value issued to the vendor per customer

B. A condition on aws:SourceIp restricting assumption to the vendor's IP ranges

C. A condition on aws:MultiFactorAuthPresent so the vendor must use MFA

D. A permissions boundary attached to the role the vendor assumes

◆ PAUSE AND DECIDE

What should the trust policy require?

A ExternalId, unique per customer

Purpose-built for exactly this. The vendor must present a secret only that customer issued, so knowing the role ARN alone is not enough.

B SourceIp restriction

The attack comes from the vendor's own platform, at the vendor's own IP addresses. An IP condition would pass.

C Require MFA

MFA is for humans. An automated SaaS platform assuming roles thousands of times cannot present an MFA code.

D Permissions boundary

Caps what the role can do once assumed. It does nothing about who may assume it, which is the actual vulnerability.

THE SIGNAL

This is the confused deputy problem described without naming it. ExternalId is the documented answer whenever a third party assumes roles across many customers.

A company runs about 400 servers in its own data centre. Each needs to write backup files to Amazon S3. Today each server holds an IAM user's access keys in a configuration file, and those keys have not been rotated in three years.

Security requires temporary, automatically rotating credentials. The servers cannot be migrated to EC2, and the company already operates its own certificate authority.

What should the company implement?

A Rotate the access keys automatically with a scheduled Lambda function

B Store the access keys in AWS Secrets Manager and have each server fetch them at startup

C Run an EC2 proxy that the on-premises servers call, using the instance profile

D Use IAM Roles Anywhere, with the existing certificate authority registered as a trust anchor

◆ PAUSE AND DECIDE

What should the company implement?

A Automated key rotation

Improves a bad pattern rather than replacing it. The keys are still long-lived between rotations and still sit in a config file.

B Keys in Secrets Manager

Circular. The server needs AWS credentials to read Secrets Manager, so it still needs a stored credential to start.

C EC2 proxy

Every backup byte now crosses through a server you must run, scale and secure, purely to avoid a credential problem with a direct solution.

D IAM Roles Anywhere

Exchanges an X.509 certificate for temporary credentials from STS. No stored keys, automatic rotation, and the CA already exists.

THE SIGNAL

Option B's circularity is the useful lesson: you cannot use an AWS service to bootstrap AWS credentials, because reaching it requires credentials already.

What to carry forward

- **Applications are given credentials, never hold them.** Every runtime has a delivery mechanism.
- **Role chaining caps at 1 hour** and fails if you ask for more. Lambda and Fargate chain by default.
- The **credential chain** order matters: the first provider to answer wins, silently.
- **ExternalId** for third-party vendors. **SourceArn** and **SourceAccount** for AWS services.
- Revoke leaked temporary credentials with a **TokenIssueTime** deny, not by deleting anything.

> ON THE EXAM
>
> Next: Amazon Cognito - user pools, identity pools, and the token exchange this module has been pointing at.